Center Weighted Convolution and GraphSAGE Cooperative Network for Hyperspectral Image Classification

Ying Cui, Chao Shao, Li Luo, Liguo Wang, *Member, IEEE*, Shan Gao, and Liwei Chen

***Abstract*— Hyperspectral image (HSI) classification is one of the basic tasks of remote sensing image processing, which is to predict the label of each HSI pixel. Convolutional neural network (CNN) and graph convolution neural network (GCN) have become the current research focus due to their outstanding performance in the field of HSI classification in recent years. However, GCN is a transductive learning method, which needs all nodes to participate in the training process to get the node embedding. Graph sample and aggregate (GraphSAGE) is an important branch of graph neural network, which can flexibly aggregate new neighbor nodes in non-Euclidean data of any structure and capture long-range contextual relationships. Superpixel-based GraphSAGE can not only integrate the global spatial relationship of data but also further reduce its computing cost. CNN can extract pixel-level features in a small area, and our center attention module (CAM) and center weighted convolution (CW-Conv) can also improve the feature extraction ability of CNN by enhancing the dominant position of target pixels. In order to make full use of the advantages of CNN and GraphSAGE, we propose a center weighted convolution and GraphSAGE (CW-SAGE) cooperative network for HSI classification. Specifically, graph simple and aggregate branch is constructed by superpixel-based encoder and decoder modules, and then, pixel-level features are extracted by a central attention CNN. Finally, the features of the two branches are spliced together for feature fusion. We conduct experiments on three hyperspectral datasets and compare the results with other current state-of-the-art methods. A series of experiments demonstrates the advantages of our method.**

***Index Terms*— Attention mechanism, convolutional neural network (CNN), feature fusion, graph sample and aggregate (GraphSAGE), hyperspectral image (HSI) classification.**

Manuscript received 3 March 2023; accepted 1 April 2023. Date of publication 5 April 2023; date of current version 18 April 2023. This work was supported in part by the Nature Science Foundation of Heilongjiang Province under Grant LH2020F021, in part by the National Natural Science Foundation of China under Grant 62071084, and in part by the Leading Talents Project of the State Ethnic Affairs Commission. *(Corresponding author: Liwei Chen.)*

Ying Cui, Chao Shao, Li Luo, Shan Gao, and Liwei Chen are with the College of Information and Communication Engineering and the Key Laboratory of Advanced Marine Communication and Information Technology, Ministry of Industry and Information Technology, Harbin Engineering University, Harbin 150001, China (e-mail: cuiying@hrbeu.edu.cn; shaochao@hrbeu.edu.cn; luoli@hrbeu.edu.cn; gaoshan08@hrbeu.edu.cn; chenliwei@hrbeu.edu.cn).

Liguo Wang is with the College of Information and Communication Engineering and the Key Laboratory of Advanced Marine Communication and Information Technology, Ministry of Industry and Information Technology, Harbin Engineering University, Harbin 150001, China, and also with the College of Information and Communication Engineering, Dalian Minzu University, Dalian 116600, China (e-mail: wangliguo@hrbeu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2023.3264653

I. Introduction

HYPERSPECTRAL image (HSI) has the advantages of high spectral resolution and continuous spectral bands, so HSIs have rich spatial and spectral information [1], [2], [3], [4]. In view of the above advantages, HSI classification technology has been widely used in aerospace telemetry, agriculture, geological exploration, and medical diagnosis. Thus, what researchers need to solve is how to accurately classify each pixel of HSI data.

In the early research stage, many machine learning methods were applied to HSI classification. The representative methods using spectral information include support vector machine (SVM) [5] and sparse representation classification [6], [7]. These classification methods are simple in calculation and do not consider the spatial information of HSI, which will lead to the Hughes effect [8]. Thus, the early HSI classification techniques have not been able to achieve satisfactory results due to their limited feature extraction capabilities. Deep learning is to learn the inherent laws and representation levels of sample data, and it can fully extract the spatial and spectral information of HSI data.

Currently, deep learning methods, such as convolutional neural network (CNN) [9], recurrent neural network (RNN) [10], generative adversarial networks (GANs) [11] [12], and stacked autoencoder (SAE) [13], have been applied to HSI classification with satisfactory results. A spectral–spatial residual network (SSRN) was designed in [14], using residual connections and 3-D CNN to alleviate the problem of gradient descent. The A2S2K-ResNet proposed by Roy et al. [15] first introduced an attention-based adaptive spectral–spatial kernel module to learn selective 3-D convolutional kernels for HSI classification and designed an improved spectral–spatial ResNet to extract spectral–spatial joint features. Meanwhile, an efficient feature recalibration (EFR) mechanism is employed to better capture the cross-channel correlation of the transformed feature maps. In order to better extract the spectral and spatial features of HSI, in [16], the dual-channel CNN-based network [two-branch deep CNN (Two-CNN)] was proposed to integrate 1-D and 2-D convolution branches to jointly extract spectral and spatial features. Cui et al. [17] proposed a dual-channel network structure to extract spatial features and spectral features and introduced a self-calibration convolution module when extracting

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

contextual features to capture domain correlations. In [18], combine 2-D CNN and 3-D CNN to learn HSI features, where 2-D CNN is used to learn spatial features and 3-D CNN is used to learn spectral–spatial joint information. The essence of the attention mechanism is a set of weight coefficients learned autonomously by the network, which is used to enhance the area of interest and suppress the irrelevant background area. The squeeze-and-excitation (SE) block [19] is a unit design to perform dynamic channelwise feature attention. Jun et al. [20] presented a dual-attention network for scene segmentation composed of position attention and channel attention working in parallel. In [21], a double-branch multi-attention (DBMA) mechanism network was proposed, which uses the convolutional block attention module [22] to integrate spectral and spatial information. Similarly, a double-branch dual-attention (DBDA) network was proposed in [23], which further integrates spectral and spatial information by using the dual-attention network mechanism network in [20]. Influenced by the natural language processing (NLP) model bidirectional encoder representation from transformers (BERT), He et al. [24] proposed HSI-BERT. The BERT-based HSI-BERT is more flexible than the CNN-based model. It can not only deal with different regions but also can be promoted from different regions. In [25], the researchers considered the problem that the improvement of classification accuracy often leads to a significant increase in model complexity and then proposed a lightweight model suitable for HSI classification based on MobileNetV3 [26].

The deep learning model based on CNN usually needs many training samples to achieve satisfactory classification accuracy, and CNN can only process Euclidean data in local regions, so the relationship of long-range context in HSI cannot be well captured. In order to improve these problems, researchers have carried out extensive classification research using graph convolution neural network (GCN) [27]. However, the node fusion efficiency of using GCN directly for HSI classification is low and the amount of computation is unbearable. Hong et al. [28] proposed to use mini-batch sampling for GCN (miniGCN) so that it can perform HSI classification training in mini-batch as same as CNN, and miniGCNs provide a new perspective to guide how to use GCN to classify HSI more effectively. Another method using superpixels instead of pixels as nodes was proposed in [29], [30], which greatly reduces the size of the graph and reduces the amount of computation. The size and shape of superpixels can be changed according to the HSI data, so superpixels can provide a good description of different classes of land cover. In [31], [32], and [33], a model that fuses GCN and CNN in one network is proposed to extract pixel-level features and long-range contextual relations. It should be noted that GCN is a transductive learning and full-graph training method, which is difficult to aggregate new nodes, and its training process needs to put the adjacency matrix and the feature matrix together in memory or display memory. The complexity of its algorithm is extremely high, and it is not desirable for large-scale graph data.

Therefore, Pan et al. [34] introduced graph sample and aggregate (GraphSAGE) [35] into HSI classification first and proposed spectral–spatial GraphSAGE (S^2GraphSAGE) algorithm uses spectral–spatial distance information to determine the adjacency relationship of the pixels image. The multi-scale graph sample and aggregation network (MSAGE-CAL) method proposed in [36] uses multiscale graph convolution to make full use of spatial information and reduce computational complexity. Later, Ding et al. [37] combined GraphSAGE and attention mechanism to propose graph sample and aggregate attention (SAGE-A), which enhanced the feature extraction ability of GraphSAGE through the attention mechanism.

CNN and GCN, as the two research hotspots of HSI classification task at present, still have a lot of room for improvement. GraphSAGE solves the problem of high computational complexity of GCN full-graph and full-batch training method so that it can learn graph structure information on large HSI dataset. For the CNN-based method, the target pixel and its adjacent pixels are usually trained as subcube sample (i.e., a patch), and its class label is the class label of its central pixel, so there is a great probability that the pixel in the central part of each patch is the same as the target pixel. If the pixel in the central part and the pixel around the patch are treated in the same way, the prediction of the target pixel may be wrong. Also, because the training is based on small patch, the CNN-based method cannot extract the long-range context of the target pixel. In addition, through reading this article, we know that the combination of CNN and GCN improves the ability of HSI classification because GCN can just extract the long-range context to make up for the deficiency of CNN. Encouraged by this, can we design a hybrid deep learning framework of CNN and GraphSAGE to extract the complete information of the target node while overcoming the shortcoming that GCN cannot be extended to large-scale dataset?

To address these issues, we propose to combine CNN and GraphSAGE into a network and extract features through two branches (CWG-SAGE). Branch 1 learns superpixel-level long-range contextual features and relations in large irregular regions via GraphSAGE, and branch 2 learns pixel-level features on small regular regions through CNN. In branch 2, we introduce a center weighted convolution (CW-Conv) kernel instead of the traditional convolution kernel and proposed a center attention module (CAM). When extracting features, let the network learn the relationship between the center pixel features and the edge features by itself so that the center pixel feature can play a leading role and the edge feature can play an auxiliary role. Finally, fuse the features extracted from the two branches. The main contributions in this article are given as follows.

1) We propose a novel deep learning framework that combines CNN and GraphSAGE through a graph encoder and decoder, which can fully exploit the advantages of CNN and GraphSAGE.
2) In view of shortcomings of CNN, we introduce CW-Conv and propose a CAM module, let the network learn the primary and secondary relationship between the center pixel feature and the edge pixel feature, and enhance the feature extraction ability.
3) We developed three schemes to fuse features extracted from CNN and GraphSAGE, including addition fusion,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

elementwise multiplication, and concatenation, and achieved better results in HSI classification.

4) The proposed deep learning network model improves feature representation and achieves the highest classification accuracy with limited training samples on three publicly available datasets (i.e., Indian Pines, Salinas, and WHU-Hi-HongHu).

The rest of this article is arranged as follows. In Section II, We introduced the research related to our work. In Section III, we introduce the structure of the CWG-SAGE model and the role of each module. In Section IV, a series of experiments is carried out to verify the effect of the model. In Section V, we verify the effectiveness of each module and the influence of various parameters on the model through a large number of analytical experiments. Finally, we conclude this article in Section VI and propose directions for future work.

II. Related Work

Many researchers have published their methods of classifying HSI. In this section, we mainly introduce the GCN method and CNN method, which have a great relationship with our work.

A. GCN Layers

For a graph $\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E})$, where $\boldsymbol{V}$ and $\boldsymbol{E}$ represent nodes and edges, respectively, the adjacency matrix $\boldsymbol{A}$ can be expressed as follows:

$$\boldsymbol{A}_{i,j} = \begin{cases} \exp\left(-\dfrac{\| \upsilon_i - \upsilon_j \|^2}{\sigma^2}\right), & \text{if } V_i \text{ and } V_j \text{ are adjancent} \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

where σ is a parameter, vectors υ_i and υ_j denote the spectral signatures of the nodes V_i and V_j, respectively, and $\boldsymbol{A}_{i,j}$ is the edge of V_i and V_j. Once $\boldsymbol{A}$ is determined, we create the corresponding graph Laplacian matrix $\boldsymbol{L} = \boldsymbol{D} - \boldsymbol{A}$, where $\boldsymbol{D}$ is the degree matrix of the graph, i.e., $\boldsymbol{D}_{i,i} = \sum_j \boldsymbol{A}_{i,j}$ [38], [39]. In order to enhance the generalization ability of the graph, the corresponding symmetrically normalized Laplacian matrix ($\boldsymbol{L}_n$) is

$$\boldsymbol{L}_n = \boldsymbol{I} - \boldsymbol{D}^{-\frac{1}{2}}\boldsymbol{A}\boldsymbol{D}^{-\frac{1}{2}} \tag{2}$$

where $\boldsymbol{I}$ is an identity matrix. Then, we perform spectral decomposition on $\boldsymbol{I}_n$, $\boldsymbol{L}_n = \boldsymbol{U}\epsilon\boldsymbol{U}^{-1}$, where $\boldsymbol{U} = (\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n)$ is the set of eigenvectors of $\boldsymbol{L}_n$ and ϵ is the diagonal matrix of eigenvalues of $\boldsymbol{L}_n$, and because $\boldsymbol{U}$ is also an orthogonal matrix, i.e., $\boldsymbol{U}\boldsymbol{U}^{-1} = \boldsymbol{U}\boldsymbol{U}^{\top} = \boldsymbol{E}$, (2) can be written as $\boldsymbol{L}_n = \boldsymbol{U}\epsilon\boldsymbol{U}^{\top}$. Then the convolution of $\boldsymbol{x}$ and $\boldsymbol{g}$ on the graph can be expressed as

$$\boldsymbol{x} * \boldsymbol{g} = \boldsymbol{U}\left[\boldsymbol{U}^{\top}\boldsymbol{x}\right] \cdot \left[\boldsymbol{U}^{\top}\boldsymbol{g}\right]. \tag{3}$$

Then, let $\boldsymbol{U}^{\top}\boldsymbol{g}$ be written as $\boldsymbol{g}_{\theta}(\epsilon)$, and the convolution on a graph can finally be expressed as

$$\boldsymbol{x} * \boldsymbol{g}_{\theta} = \boldsymbol{U}\boldsymbol{g}_{\theta}(\epsilon)\boldsymbol{U}^{\top}\mathbf{x} \tag{4}$$

where g_{θ} can be understood as the function of the eigenvalues (ϵ) of $\boldsymbol{L}_n$, that is, $\mathbf{g}_{\theta}(\epsilon)$. To solve the problem that the computation of (4) is too complicated, Hammond et al. [40] approximately fit $\boldsymbol{g}_{\theta}(\epsilon)$ by using the Kth order truncated expansion of the Chebyshev polynomial. Thus, (4) can be written as

$$\boldsymbol{x} * \boldsymbol{g}_{\theta} \approx \sum_{k=0}^{K} \theta'_k \boldsymbol{T}_k\left(\widetilde{\boldsymbol{L}}_n\right)\boldsymbol{x} \tag{5}$$

where θ'_k is the Chebyshev coefficients, $\boldsymbol{T}_k$ is the Chebyshev polynomials, and $\widetilde{\boldsymbol{L}}_n$ denotes the normalized $\boldsymbol{L}_n$. Further simplification (5), when $K = 1$, Kipf and Welling [27] further approximated the largest eigenvalue ϵ_{max} to 2; therefore, (5) can be simplified to

$$\boldsymbol{x} * \boldsymbol{g}_{\theta} \approx \theta(\boldsymbol{I}_N + \boldsymbol{L}) = \theta\left(\boldsymbol{I} + \boldsymbol{D}^{-\frac{1}{2}}\boldsymbol{A}\boldsymbol{D}^{-\frac{1}{2}}\right). \tag{6}$$

Finally, we get the propagation rules for GCN as follows:

$$\boldsymbol{H}^{(l+1)} = \sigma\left(\widetilde{\boldsymbol{D}}^{-\frac{1}{2}}\widetilde{\boldsymbol{A}}\widetilde{\boldsymbol{D}}^{-\frac{1}{2}}\boldsymbol{H}^{(l)}\boldsymbol{W}^{(l)} + \boldsymbol{b}^{(l)}\right) \tag{7}$$

where $\widetilde{\boldsymbol{A}} = \boldsymbol{A} + \boldsymbol{I}$; $\widetilde{\boldsymbol{D}}$ represents the degree matrix of $\widetilde{\boldsymbol{A}}$; σ is the activation function; and $\boldsymbol{H}^{(l)}$, $\boldsymbol{W}^{(l)}$, and $\boldsymbol{b}^{(l)}$ represent the output values, weight matrix, and biases of the lth layer, respectively.

B. CNN Layers

CNN is a common deep learning architecture, inspired by biological natural visual cognitive mechanism. Because of its structural characteristics of local area connection and weight sharing, it makes the CNN perform well in the field of image processing. These two characteristics of CNN reduce the complexity of the network model, reduce the number of weights, and have strong generalization ability. Therefore, CNN also plays an important role in HSI classification.

In CNN, convolution is realized by calculating the sum of the dot products between the input data and the kernel. We usually call the convolution with the convolution kernel size of 1 × 1.1-D CNN. Due to the use of the minimum window, 1-D CNN loses the function of convolution layer to recognize the patterns formed by the adjacent elements of high and wide dimensions. In fact, the main calculation of 1-D CNN takes place in the channel dimension, that is, the spectral characteristics of HSI can be extracted through 1-D CNN. Similar to 1-D CNN, the convolution kernel of 2-D has spatial dimensions so that spatial information can be extracted. Therefore, we usually use 1-D CNN and 2-D CNN to extract spectral information and spatial information of HSI, respectively. The 2-D CNN can be expressed as

$$\upsilon_{i,j}^{x,y} = \phi\left(\sum_{\tau=1}^{d_{l-1}} \sum_{\rho=-\gamma}^{\gamma} \sum_{\sigma=-\delta}^{\delta} K_{i,j,\tau}^{\sigma,\rho} \times \upsilon_{i-1,\tau}^{x+\sigma,y+\rho} + b_{i,j}\right) \tag{8}$$

where $\upsilon_{i,j}^{x,y}$ is the activation value at spatial position (x, y) in the jth feature map of the ith layer, ϕ is the activation function, $b_{i,j}$ is the bias parameter for the jth feature map of the ith layer, d_{l-1} is the number of feature map in the $(l-1)$th layer, $2\gamma + 1$ is the width of kernel, $2\rho + 1$ is the height of kernel, and $w_{i,j}$ is the value of weight parameter for the jth feature map of the ith layer. In addition, when the width and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

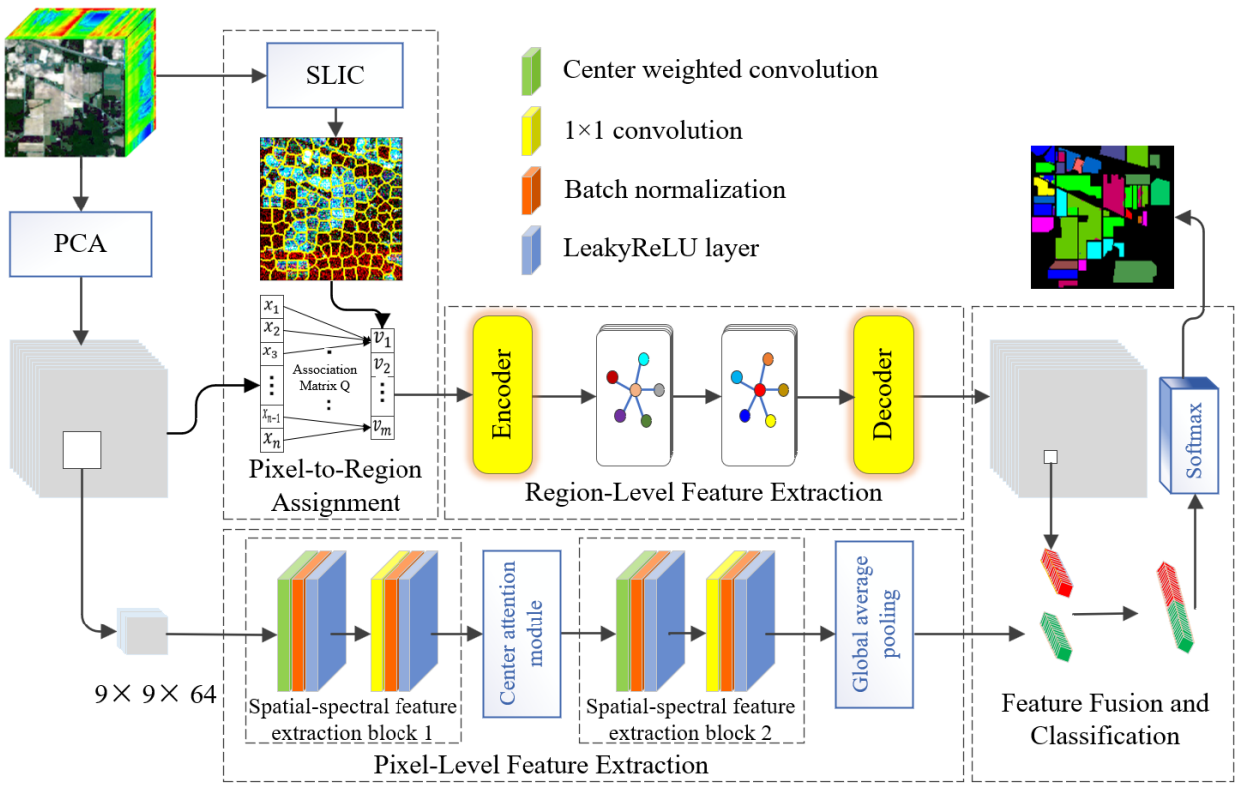


Fig. 1. Overall architecture of the proposed CWG-SAGE.

height of the convolution kernel are both 1, it is 1-D CNN, so it will not be repeated.

III. Proposed Method

In this section, we will introduce in detail CWG-SAGE for HSI classification (Fig. 1). It mainly consists of four parts, pixel-to-region assignment, pixel-level feature extraction, region-level feature extraction, feature fusion, and classification module. The original HSI data are divided into superpixels by simple linear iterative clustering (SLIC) [41] in the pixel-to-region assignment module to obtain superpixels to construct a graph encoder. Then, the network extracts region-level features through branch 1 and pixel-level features through branch 2. The obtained features are fused, and finally, HSI classification is performed.

A. Pixel-to-Region Assignment

If each pixel of the HSI is regarded as a node of the graph, the computational load and computational difficulty will be unbearable. On this issue, we find that there is a high probability that adjacent pixels belong to the same land cover type. Therefore, SLIC is used to segment the HSI into a small number of local regions, each of which is composed of pixels with strong spectral–spatial similarity. Specifically, SLIC performs image region segmentation by iteratively increasing local clustering using the k-means algorithm. In this article, local regions are used as nodes of the graph, which can significantly reduce the number of nodes of the graph and improve the computational efficiency, and inspired by [31], we apply a structure converter to allow this feature to propagate between pixels and superpixels.

The HSI has high spectral resolution, strong correlation between its bands, and large correlation coefficient between the bands, which is easy to cause hyperspectral redundant information stacking. Principal component analysis (PCA) can just alleviate the problem of dimensional disaster while minimizing the information loss. Therefore, we first remove the redundant spectrum of HSI based on PCA. We denote $\boldsymbol{X} \in R^{H\times W\times B}$ as the HSI after PCA and $\boldsymbol{Q} \in R^{HW\times Z}$ as the association matrix between pixels and superpixels in the structure converter, where Z in $\boldsymbol{Q}$ represents the number of superpixels; then, we have

$$\boldsymbol{Q}_{i,j} = \begin{cases} 1, & \text{if } \bar{\boldsymbol{X}}_i \in S_j \\ 0, & \text{if } \bar{\boldsymbol{X}}_i \notin S_j \end{cases} \quad \bar{\boldsymbol{X}} = \text{Flatten}(\boldsymbol{X}) \tag{9}$$

where Flatten$(\cdot)$ denotes the HSI data flattened by the spatial dimension, $\boldsymbol{Q}_{i,j}$ denotes the value of $\boldsymbol{Q}$ at location (i,j), $\bar{\boldsymbol{X}}_i$ denotes the ith pixel in $\bar{\boldsymbol{X}}$, and S_j denotes the jth superpixels. In the pixel-to-region assignment module of Fig. 1, we can more directly see the process of generating the feature vector of the superpixel node through the correlation matrix $\boldsymbol{Q}$, and the HSI composed of superpixels after SLIC. Then, we can use the following formula to achieve feature conversion as:

$$\boldsymbol{V} = \text{Encoder}(\boldsymbol{X};\boldsymbol{Q}) = \widehat{\boldsymbol{Q}}^T \text{Flatten}(\boldsymbol{X}) \tag{10}$$

$$\widetilde{\boldsymbol{X}} = \text{Decoder}(\boldsymbol{V};\boldsymbol{Q}) = \text{Reshape}(\boldsymbol{Q}\boldsymbol{V}) \tag{11}$$

where $\widehat{\boldsymbol{Q}}$ denotes the normalized $\boldsymbol{Q}$ by column, and the reason for the normalization operation is that the feature of a

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 GraphSAGE Embedding Generation (i.e., Forward Propagation) Algorithm

Input: Graph $\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E})$; input features $\{\boldsymbol{x}_v, \forall v \in \boldsymbol{V}\}$; the number of layers of the network K; weight matrices $\boldsymbol{W}^K$; $\forall\ k \in \{1, \ldots, K\}$; non-linearity σ; differentiable aggregator functions AGG; neighhood function $N{:}v \to 2^v$

Output: Vector representations $\boldsymbol{z}_v$ for all $v \in \boldsymbol{V}$

$h_v^0 \leftarrow \boldsymbol{x}_v, \forall v \in \boldsymbol{V}$
for $k = \{1, \ldots, K\}$ **do**
  for $v \in V$ **do**
    $\boldsymbol{h}_{N(v)}^k \leftarrow AGG(\{\boldsymbol{h}_u^{k-1}, \forall u \in \boldsymbol{N}(v)\})$;
    $\boldsymbol{h}_v^k \leftarrow \sigma(\boldsymbol{W}^k \cdot CONCAT(\boldsymbol{h}_v^{k-1}, \boldsymbol{h}_{N(v)}^k))$
  end for
  $\boldsymbol{h}_v^k \leftarrow \boldsymbol{h}_v^k / \parallel \boldsymbol{h}_v^k \parallel_2, v \in V$
end for
return $\boldsymbol{z}_v \leftarrow \boldsymbol{h}_v^K, v \in V$

superpixel is the mean of its subpixel features, $\boldsymbol{V}$ denotes the nodes composed of superpixels, and Reshape denotes restoring the spatial dimension of the flattened data. $\widetilde{\boldsymbol{X}}$ denotes the features after assignment from regions to pixels.

B. Region-Level Feature Extraction

From the GCN layers of related work, we can see that as a transductive learning method, GCN requires all graph nodes to participate in node embedding at the same time in the training process. It may bring huge computational costs to HSI with a large number of pixels. Besides, the graph structure of GCN is fixed, so it cannot be generalized to unknown nodes naturally. In response to these problems, researchers proposed graph sample and aggregate (GraphSAGE) in [34], and GraphSAGE is an inductive learning framework that can efficiently generate unknown vertex embeddings using vertex attribute information. Its core idea is to generate the embedding vector of the target vertex by learning a function that aggregates the adjacent vertices. It overcomes the limitation of memory and video memory of GCN in the training process by training in batches and requiring only neighbor nodes to participate in the calculation. Algorithm 1 describes the embedding generation process.

In Algorithm 1, K represents the number of hops for each vertex to aggregate neighbor nodes, and it can also represent the number of network layers. $\boldsymbol{W}^K$ represents the weight matrix that needs to be learned in the network. $\boldsymbol{N}(v)$ represents the neighbor node feature set used to fuse the target node information, $\boldsymbol{h}_v^{k-1}$ represents the node features in the $k-1$ layer, and $\boldsymbol{h}_v^k$ represents the node features in the k layer. In this algorithm, the selection of neighbor nodes for fusing the target node information is randomly selected from all neighbor nodes of the target node. This is for a large number of neighbor nodes in the citation network data. However, this does not apply to the HSI dataset because the number of neighbor nodes of the graph nodes formed after SLIC is very small, so we propose to select all neighbor nodes of the target node for feature fusion instead of randomly selecting them. This approach also prevents a feature loss due to random selection.

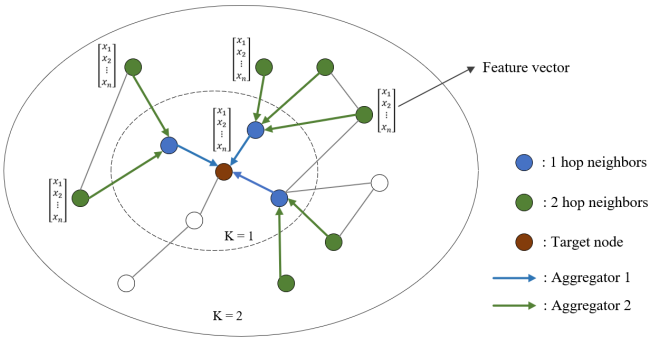


Fig. 2. Visual illustration of the GraphSAGE sample and aggregate approach. The information of the target node is aggregated by its neighbor nodes, and it does not need to consider all nodes of the graph, so the generalization performance of this transductive learning framework is better than the traditional GCN framework.

Thus, for the neighbor node feature aggregation of node i, it can be expressed as

$$\boldsymbol{h}_v^k \leftarrow \sigma\left(\boldsymbol{W}^k \cdot \mathrm{CONCAT}\left(\boldsymbol{h}_v^{k-1}, \boldsymbol{h}_{N(v)}^k\right)\right) \tag{12}$$

$$\boldsymbol{h}_{N(v)}^k = \mathrm{MEAN}\left(\boldsymbol{h}_u^{k-1}\right) = \frac{\sum_{v=1}^{n} \boldsymbol{h}_u^{k-1} \cdot \boldsymbol{A}_{i,v}}{\sum_{v=1}^{n} \boldsymbol{A}_{i,v}}. \tag{13}$$

$\mathrm{CONCAT}(\cdot, \cdot)$ means to concatenate two feature vectors together, $MEAN(\cdot)$ means taking the mean value of the n neighbor node features, and $\boldsymbol{A}_{i,v}$ means the edge of target node i and its adjacent nodes.

Fig. 2 shows the GraphSAGE neighbor features aggregation process in detail. The small brown circles, the small blue circles, and the small green circles represent the target node, the one-hop neighbor nodes, and the two-hop neighbor nodes, respectively. The feature information of the two-hop neighbor nodes is aggregated to the one-hop neighbor node in the direction of the green arrow, and the feature information of the one-hop neighbor nodes is aggregated to the target node in the direction of the blue arrow.

In this branch, the spectral feature dimension after PCA dimensionality reduction is 64. The feature dimension after one-hop neighbor node information aggregation and two-hop neighbor node information aggregation are both 64.

C. Pixel-Level Feature Extraction

Due to the superior ability of CNN to extract local spatial and spectral features, we extract HSI pixel-level features through CNN. As shown in the pixel-level feature extraction module of Fig. 1, there are mainly two spatial–spectral feature extraction blocks. Each spatial–spectral feature block extracts spatial features by CW-Conv, and spectral features are extracted by 1-D CNN and then use the CAM module to enhance the feature extraction ability. Meanwhile, to make the final input more stable, we apply batch normalization (BN) [42] layer in our CWG-SAGE.

1) Center Weighted Convolution: Considering the diversity of edge data and the redundant data filled in padding, therefore, when extracting edge features and central features, the relationship between them should be reconsidered. We propose to increase the weight of its center pixel feature; to this end,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

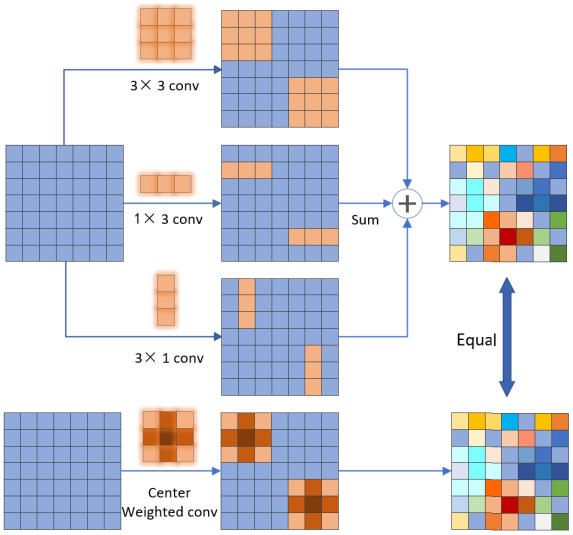


Fig. 3. Structure of CW-Conv. Here, we have three convolutional layers with kernel sizes 3 × 3, 1 × 3, and 3 × 1; the sum of the results of these three convolutional layers can be equivalent to the output of the CW-Conv kernel by exploiting the additivity of the convolution kernel. The schematic only depicts the top-left and bottom-right convolution operations.

we design the CAM module; and inspired by [43], we use CW-Conv kernel instead of the traditional convolution kernel. The structure of CW-Conv is shown in Fig. 3, and its essence is to use the additivity of convolution to build an asymmetric convolution network. We use sliding windows to demonstrate some intuitive additivity of 2-D CNN with different kernel sizes. Here, we use three convolutional layers with kernel sizes of 3 × 3, 1 × 3, and 3 × 1, and these three convolutional layers take the same input. In the figure, we only show the top-left and bottom-right sliding windows. It can be noticed that the sliding windows of channels 1–3 are the same, which is also the basis for achieving additivity. Thus, the result of adding the outputs of the three channels is the same as adding the three convolution kernels and then convolving. Its process can be expressed as follows:

$$I * K^{(1)} + I * K^{(2)} + I * K^{(3)} = I * \left(K^{(1)} \oplus K^{(2)} \oplus K^{(3)}\right) \tag{14}$$

where I represents the input data, $K^{(\cdot)}$ represents the convolution kernels of different sizes, and $\oplus$ represents the element-by-element addition of the kernel parameters at the corresponding positions.

2) Center Attention Module: In the HSI classification task, the input is cube data, but only its center pixel category prediction, so we should give more weight to the center pixel and suppress other interference pixels to ensure the correct prediction of pixel category. Therefore, we designed the CAM, which is shown in Fig. 4.

The specific operation of CAM is to intercept the 3 × 3 region from the middle of the input data as a convolution kernel and then perform the convolution operation on the input data to obtain the similarity with the pixels of the central region. However, the convolution output value here will be very large, so we need to normalize and then weight the input data. We denote $F \in R^{H\times W\times C}$ as input and $F_C \in R^{3\times 3\times C}$

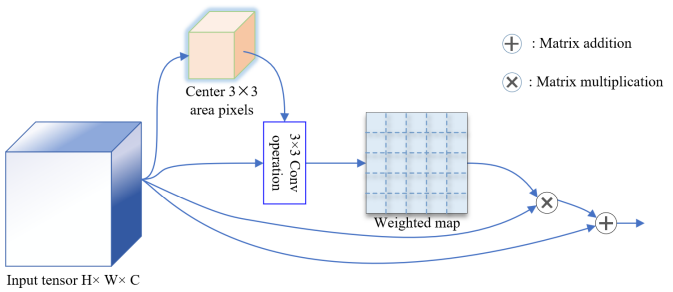


Fig. 4. Detailed structure display of the proposed CAM. We take the 3 × 3 region in the middle of the input data as the convolution kernel to perform convolution operation with the input data to obtain the relationship between edge pixels and middle key pixels.

as the center pixels, where H, W, and C denote the height, weight, and the number of channels of F, respectively. Thus, the CAM can be defined as

$$F_O = \alpha(\sigma(\varphi(F_C, F)) \otimes F) + F \tag{15}$$

where φ means the convolution operation, σ stands for normalization, $\otimes$ denotes the rescaling of input, and F_O is the output.

The input of the pixel-level feature extraction branch is a 9 × 9 × 64 patch. To keep consistent with the feature dimensions obtained by the region-level feature extraction, we set the number of convolution kernels of CW-Conv and 1-D CNN to 64, so the feature dimension obtained in the pixel-level feature extraction branch is 64. After the global average-pooling module, a feature vector with a dimension of 64 is obtained.

D. Feature Fusion and Classification

Decoding the features after GraphSAGE can get the region-level features with the target node dimension of 64, and the feature dimension of the node obtained through the pixel-level feature extractor is also 64. Three fusion strategies are considered, addition (A), elementwise multiplication (M), and concatenation (C). The three fusion models (A, M, and C) can be represented as follows:

$$\boldsymbol{H}_A = \boldsymbol{H}_{\text{CNN}} \oplus \boldsymbol{H}_{\text{GraphSAGE}} \tag{16}$$

$$\boldsymbol{H}_M = \boldsymbol{H}_{\text{CNN}} \odot \boldsymbol{H}_{\text{GraphSAGE}} \tag{17}$$

$$\boldsymbol{H}_C = \left[\boldsymbol{H}_{\text{CNN}}, \boldsymbol{H}_{\text{GraphSAGE}}\right] \tag{18}$$

where operators $\oplus$, $\odot$, and $[,]$ denote elementwise addition, elementwise multiplication, and concatenation, respectively, and $\boldsymbol{H}_{\text{CNN}}$ and $\boldsymbol{H}_{\text{GraphSAGE}}$ are represented as the features extracted from branches 1 and 2, respectively. The fusion strategy is concatenation in this article. Finally, the probability of each category label is obtained by the softmax function. Here, we use the cross-entropy loss function as the loss function to train the network, and it can be written as

$$L(\boldsymbol{Y}, \boldsymbol{P}) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log\left(p_{i,c}\right) \tag{19}$$

where $\boldsymbol{Y}$ denotes the right label, $\boldsymbol{P}$ denotes the predicted value of each pixel, and $y_{i,c}$ denotes the cth element of the label $\boldsymbol{Y}$.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

$p_{i,c}$ denotes the probability of the pixel i belonging to the cth class. C and N refer to the total number of categories and samples of the training datasets, respectively.

IV. Experimental Results

In this section, to evaluate the effectiveness of CWG-SAGE, we compare the CWG-SAGE model with hybrid spectral CNN (HybridSN) [18], double-branch multi-attention mechanism network (DBMA) [21], DBDA mechanism network [23], SAGE-A [37], CNN-enhanced GCN (CEGCN) [31], weighted feature fusion of CNN and graph attention network (WFCG) [32], and fast dynamic graph convolutional network and CNN (FDGC) [33]. For a fair comparison, all experiments are performed in the same environment using the hyperparameters in the original paper.

Furthermore, in order to quantitatively and qualitatively evaluate the classification performance of each method, four quantitative evaluation metrics were employed in this experiment: per-class accuracy, overall accuracy (OA), average accuracy (AA), and kappa coefficient (kappa). The specific definitions and calculation methods of OA, AA, and kappa are given as follows:

$$\mathrm{OA} = \frac{\mathrm{IM}}{\mathrm{IN}} \tag{20}$$

$$\mathrm{AA} = \frac{\sum_{k=1}^{K} \frac{\mathrm{IM}_k}{\mathrm{IN}_k}}{K} \tag{21}$$

$$\mathrm{kappa} = \frac{\mathrm{IM} \times \mathrm{IN} - \sum_{k=1}^{K} \mathrm{IP}_k \times \mathrm{IN}_k}{\mathrm{IN} \times \mathrm{IN} - \sum_{k=1}^{K} \mathrm{IP}_k \times \mathrm{IN}_k}. \tag{22}$$

IM represents the number of all correctly classified test samples, IN represents the total number of test samples, IM_k represents the number of correctly classified test samples of class k, IN_k represents the total number of test samples of class k, and IP_k represents the number of predicted test samples of class k. All experimental results are the average of ten independent runs.

TABLE I
Categories Information and Dataset Division for the Indian Pines

No	Class	Total	Train	Val	Test
1	Alfalfa	46	3	1	42
2	Corn-notill	1428	72	15	1341
3	Corn-mintill	830	42	9	779
4	Corn	237	12	3	212
5	Grass-pasture	483	25	5	453
6	Grass-trees	730	37	8	685
7	Gress-pasture-mowed	28	2	1	25
8	Hay-windrowed	478	24	5	449
9	Oats	20	1	1	18
10	Soybean-notill	972	49	10	913
11	Soybean-mintill	2455	123	25	2307
12	Soybean-clean	593	30	6	557
13	Wheat	205	11	3	191
14	Woods	1265	64	13	1188
15	Buildings-Grass-Trees-Drives	386	20	4	362
16	Stone-Steel-Towers	93	5	1	87
	Total	10249	520	110	9619

TABLE II
Categories Information and Dataset Division for the Salinas

No	Class	Total	Train	Val	Test
1	Brocoli_green_weeds1	2009	21	21	1967
2	Brocoli_green_weeds_2	3726	38	38	3650
3	Fallow	1976	20	20	1936
4	Fallow_rough_plow	1394	14	14	1366
5	Fallow_smooth	2678	27	27	2624
6	Stubble	3959	40	40	3879
7	Celery	3579	36	36	3507
8	Grapes_untrained	11271	113	113	11045
9	Soil_vinyard_develop	6203	63	63	6077
10	Corn_senesced_green_weeds	3278	33	33	3212
11	Letture_romaine_4wk	1068	11	11	1046
12	Letture_romaine_5wk	1927	20	20	1887
13	Letture_romaine_6wk	916	10	10	896
14	Letture_romaine_7wk	1070	11	11	1048
15	Vinyard_untrained	7268	73	73	7122
16	Vinyard_vertical_trellis	1807	19	19	1769
	Total	54129	549	549	53031

A. Dataset Description

We validate our model using three HSI datasets in this article, the Indian Pines, the Salinas, and the WHU-Hi-HongHu [44], [45] datasets.

1) Indian Pines: The Indian Pines dataset was captured by the Airborne Visible Infrared Imaging Spectrometer (AVIRIS) [46] sensor in over the Indian Pines test site in Northwestern Indiana. It contains 145 × 145 pixels and 224 spectral reflectance bands in the wavelength range from 0.4 to 2.5 μm, with a total of 10 249 labeled pixels that can be used for classification, and these labels have 16 categories. After removing 24 noisy and water absorption bands, 200 bands are reserved. Table I list the categories, pixel counts, and dataset division of this HSI dataset.

2) Salinas: The Salinas dataset was obtained by the AVIRIS spectrometer over the Salinas Valley in California. AVIRIS acquires data in 224 bands with a center wavelength ranging from 0.4 to 2.5 μm. It has a high spatial resolution of 3.7 m. After removing water absorption and low SNR bands, 204 bands were used for analysis. It contains 512 × 217 pixels, with a total of 54 129 labeled pixels that can be used for classification, and these labels have 13 categories. Table II lists the categories, pixel counts, and dataset division of this HSI dataset.

3) WHU-Hi-HongHu: It is collected by a Headwall Nano-Hyperspec imaging sensor equipped on a DJI Matrice 600 Pro unmanned aerial vehicle platform in Honghu City, Hubei province, China. The experimental area is a complex agricultural scene with many classes of crops, and different cultivars of the same crop are also planted in the region, including Chinese cabbage and cabbage, and Brassica chinensis and small Brassica chinensis. We intercepted the region with a size of 470 × 475 pixels, which is the area with many types of ground objects and complex structures. It contains 470 × 475 pixels and 270 bands, with a total of 196 271 labeled pixels that can be used for classification, and these labels have 19 categories. Table III lists the categories, pixel counts, and dataset division of this HSI dataset.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

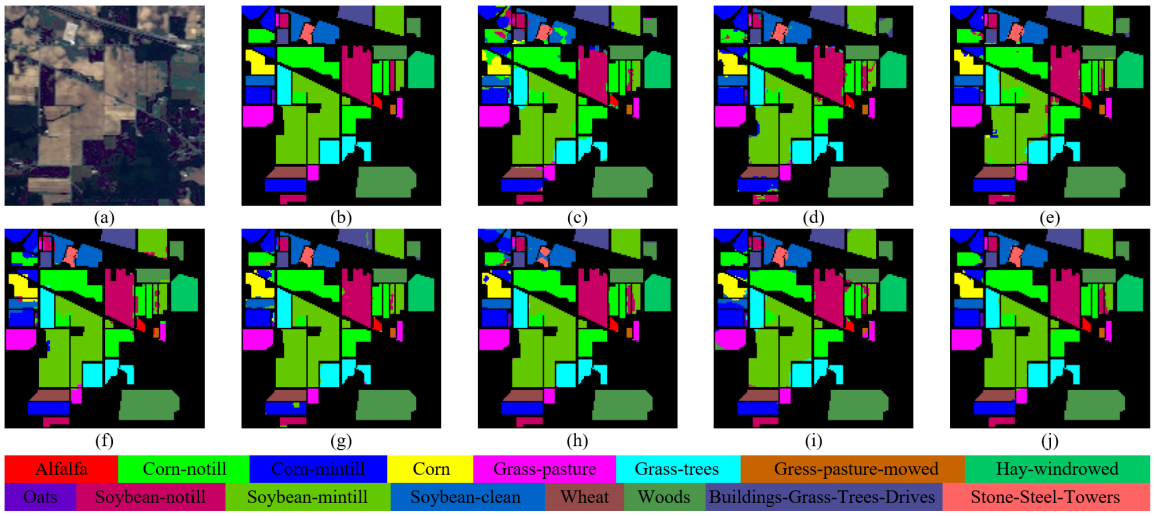


Fig. 5. Classification maps of different methods for the Indian Pines dataset. (a) False-color image. (b) Ground truth. (c) HybirdSN. (d) DBMA. (e) DBDA. (f) SAGE-A. (g) CEGCN. (h) WFCG. (i) FDGC. (j) CWG-SAGE.

TABLE III
CATEGORIES INFORMATION AND DATASET DIVISION FOR THE WHU-HI-HONGHU

No	Class	Total	Train	Val	Test
1	Red roof	3320	17	4	3299
2	Road	1609	9	2	1598
3	Bare soil	21821	110	22	21698
4	Cotton	19794	99	20	19675
5	Rape	41523	208	42	41273
6	Chinese cabbage	17862	90	18	17754
7	Pakchoi	4054	21	5	4028
8	Cabbage	10306	52	11	10243
9	Tuber mustard	12394	62	13	12319
10	Brassica parachinensis	6547	33	7	6507
11	Brassica chinensis	8587	43	9	8535
12	Small Brassica chinensis	18920	95	19	18806
13	Lactuca sativa	7356	37	8	7311
14	Celtuce	1002	6	2	994
15	Film covered lettuce	7262	37	8	7217
16	Romaine lettuce	3010	16	4	2990
17	Carrot	3217	17	4	3196
18	White radish	4201	22	5	4174
19	Garlic sprout	3486	18	4	3464
	Total	196271	992	207	195072

B. Experimental Setting

For the Indian Pines dataset, we randomly select 5% of the samples for training, 1% for validation, and the rest for testing. Considering that there are enough labeled samples in the WHU-Hi-HongHu and Salinas datasets, for the Salinas dataset, 1% of the samples are randomly selected for training, 1% for verification, and the rest for testing. For the WHU-Hi-HongHu dataset, 0.5% of the samples are randomly selected for training, 0.1% for verification, and the rest for testing. The superpixel segmentation scales in SLIC for the three datasets are set to 100, 100, and 200. The learning rate is set to 0.0005 in the Adam optimizer, and the number of training iterations is 200.

The proposed algorithm is implemented with Python3.8 and Pytorch1.7.0. The hardware used for training is an R7-5800H CPU and a NVIDIA GeForce RTX 3060 Laptop graphical processing unit (GPU).

C. Classification Maps and Comparison of Classification Performance

The classification results of different methods on each dataset are detailed in Tables IV–VI, and the classification maps obtained by these methods are shown in Figs. 5–7.

1) Result Analysis for the Indian Pines Dataset: It can be seen in Table IV and Fig. 5 that our proposed CWG-SAGE has achieved satisfactory results. Despite insufficient training samples and unbalanced sample class distribution, CWG-SAGE significantly outperforms other methods in OA, AA, and Kappa. In particular, our proposed CWG-SAGE can achieve 81.11% accuracy with so few labeled samples of class 9, which also proves the effectiveness of CWG-SAGE. HybiedSN adopts the hybrid architecture of 2-D CNN and 3-D CNN to have strong feature extraction ability, which improves the classification accuracy to a certain extent. DBMA and DBDA perform better because both channels used to extract spectral and spatial features use an attention mechanism. SAGE-A uses graphSAGE to aggregate context and refine the characteristics of graph nodes in combination with graph attention mechanism but cannot learn local pixel-level features, so its performance is not ideal. By combining CNN and GCN into one network, CEGCN takes advantage of the two algorithms of CNN and GCN, so the CEGCN algorithm is also very competitive. WFCG adds an attention mechanism based on CEGCN and reconstructs a data fusion strategy, which makes the WFCG algorithm achieve better classification results in small categories. FDGC has advantages in algorithm running speed, but it is not good in classification results.

2) Result Analysis for the Salinas Dataset: As shown in Table V and Fig. 6, for the Salinas dataset, under 1% of training samples, the classification result OA of all algorithms exceeds 95%. With the help of attention mechanism, DBMA and DBDA have obtained exciting classification results. From

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
CLASSIFICATION RESULTS FOR THE INDIAN PINES DataSET. BOLD NUMBERS INDICATE THE BEST PERFORMANCE IN THIS CATEGORY

Class	HybirdSN	DBMA	DBDA	SAGE-A	CEGCN	WFCG	FDGC	CWG-SAGE
1	71.43±23.653	**97.62±2.777**	94.44±3.521	82.73±9.278	81.40±10.647	97.21±1.741	85.12±7.865	97.14±0.952
2	89.86±7.198	88.39±7.415	93.02±2.028	92.61±1.654	96.28±1.557	95.75±2.138	93.78±3.062	**98.94±0.617**
3	91.27±3.553	95.33±1.569	**98.70±0.808**	94.57±2.471	96.25±0.633	97.59±1.390	95.73±2.638	97.56±0.936
4	88.29±7.146	89.95±5.535	94.37±4.869	88.73±6.051	91.99±11.141	**97.86±1.838**	91.07±6.022	93.51±7.939
5	95.81±1.655	92.43±3.131	96.81±2.425	93.54±7.216	97.05±1.071	97.30±1.769	91.76±5.338	**97.48±1.845**
6	98.67±1.221	97.79±0.699	96.80±1.047	**99.70±1.328**	99.38±0.756	98.31±0.729	97.17±1.874	99.24±0.549
7	55.00±35.369	68.00±17.756	**86.67±4.120**	80.00±9.797	53.69±14.509	85.38±12.259	82.22±7.372	84.00±11.593
8	99.89±0.193	99.77±0.326	**100.00±0**	98.99±0.334	**100.00±0**	**100.00±0**	97.88±2.073	99.96±0.089
9	11.11±9.271	70.59±19.563	66.67±28.598	48.61±9.918	30.31±23.54	73.68±38.09	74.62±28.326	**81.11±17.069**
10	89.16±5.319	91.92±2.839	96.98±1.705	89.40±2.074	**97.73±2.358**	96.47±2.094	92.12±2.626	96.06±2.135
11	93.54±1.487	96.80±1.125	**98.54±1.807**	97.41±0.695	98.50±0.543	98.16±1.371	97.12±1.234	98.47±0.633
12	80.61±3.213	96.01±1.912	95.16±0.883	85.81±3.413	**98.92±1.075**	98.45±1.371	87.65±3.939	98.03±1.423
13	98.41±2.565	99.47±1.212	93.65±2.262	97.91±3.627	**99.82±0.463**	99.68±0.622	98.76±1.202	98.53±0.513
14	97.47±1.218	99.49±0.304	98.81±0.774	98.86±0.717	99.58±0.491	99.63±0.223	99.08±0.278	**99.97±0.067**
15	94.75±4.027	84.36±6.092	97.86±1.195	98.48±0.569	93.58±3.404	98.55±1.942	94.97±4.255	**98.62±0.801**
16	96.55±2.629	83.72±7.233	88.76±4.052	91.66±5.474	95.74±2.159	98.60±1.356	75.45±25.809	**98.85±1.781**
OA(%)	92.56±2.231	94.53±1.122	96.83±0.394	94.91±0.355	97.49±0.472	97.82±0.408	95.02±0.679	**98.29±0.542**
AA(%)	81.91±3.655	90.73±1.865	93.51±1.417	89.94±0.731	89.39±2.328	95.79±0.314	90.91±2.899	**96.09±1.262**
Kappa	91.51±2.561	93.73±1.579	96.38±0.447	94.19±0.407	97.14±0.541	97.52±0.465	94.33±0.781	**98.05±0.618**

TABLE V
CLASSIFICATION RESULTS FOR THE SALINAS DataSET. BOLD NUMBERS INDICATE THE BEST PERFORMANCE IN THIS CATEGORY

Class	HybirdSN	DBMA	DBDA	SAGE-A	CEGCN	WFCG	FDGC	CWG-SAGE
1	99.62±0.366	**100.00±0**	**100.00±0**	**100.00±0**	**100.00±0**	99.38±0.704	99.95±0.081	**100.00±0**
2	99.97±0.027	**100.00±0**	**100.00±0**	99.90±0.166	**100.00±0**	**100.00±0**	99.86±0.234	**100.00±0**
3	98.37±2.812	99.98±0.021	99.95±0.082	99.96±0.042	99.95±0.103	99.92±0.144	99.75±0.217	**100.00±0**
4	98.97±0.741	99.60±0.389	99.41±0.585	98.93±0.754	99.61±0.345	99.31±0.528	99.26±0.682	**99.91±0.061**
5	**99.26±0.980**	97.42±1.053	98.49±0.925	97.21±0.687	98.58±0.841	99.23±0.610	98.02±1.179	98.96±1.016
6	99.81±0.297	**100.0±0**	99.93±0.134	99.82±0.114	**100.00±0**	99.85±0.229	99.68±0.471	99.98±0.033
7	99.74±0.263	99.96±0.068	99.97±0.031	99.88±0.056	**100.00±0**	**100.00±0**	99.55±0.233	99.94±0.071
8	98.24±1.371	95.85±0.855	97.06±1.194	98.52±0.110	98.89±0.901	98.74±1.127	97.32±2.288	**99.29±0.513**
9	**100.00±0**	**100.00±0**	**100.00±0**	99.98±0.013	**100.00±0**	**100.00±0**	**100.00±0**	**100.00±0**
10	98.50±1.035	97.91±1.996	98.97±1.039	98.20±1.335	98.57±1.119	98.09±1.673	97.78±1.422	**99.34±0.826**
11	99.43±0.576	99.75±0.451	**100.00±0**	97.51±1.335	99.89±0.141	99.98±0.038	99.24±0.778	**100.00±0**
12	99.60±0.265	99.97±0.043	99.95±0.085	97.35±1.849	**100.00±0**	**100.00±0**	99.74±0.331	99.99±0.02
13	99.58±0.723	99.93±0.134	99.95±0.089	93.57±4.399	**100.00±0**	99.95±0.089	98.06±1.190	99.89±0.193
14	97.02±2.247	97.21±4.410	**98.54±1.977**	93.63±3.433	98.40±1.829	98.03±1.296	98.04±1.530	98.52±1.614
15	97.99±1.071	94.20±0.787	96.95±1.079	**99.45±0.241**	99.26±0.525	99.05±0.861	99.42±1.014	99.37±0.573
16	**99.91±0.116**	99.33±0.452	99.32±0.939	99.24±0.462	99.62±0.655	99.78±0.209	98.63±0.851	99.86±0.185
OA(%)	99.02±0.425	98.01±0.325	98.76±0.242	98.90±0.152	99.45±0.213	99.35±0.251	98.91±0.397	**99.63±0.154**
AA(%)	99.13±0.426	98.82±0.257	99.28±0.151	98.24±0.243	99.54±0.074	99.45±0.140	99.02±0.276	**99.69±0.103**
Kappa	98.90±0.474	97.78±0.362	98.62±0.269	98.77±0.168	99.39±2.372	99.28±0.280	98.77±0.442	**99.59±0.172**

these two datasets, we can see that the OA of DBDA is higher than that of DBMA. Through reading the original paper, we know that the network framework of DBDA and DBMA is the same. DBDA changed the attention mechanism of DBMA from CBAM to DAN, which also shows that the effect of using DAN is better than using CBAM. It is worth noting that the classification results of HybirdSN are significantly superior to DBMA and DBDA on the Salinas dataset, which shows that HybirdSN's 2-D CNN and 3-D CNN hybrid architecture has unique advantages in processing the Salinas dataset. In addition, the classification performance of WFCG on the Salinas dataset is not as good as that of CEGCN, which indicates that the data fusion strategy of WFCG is not applicable to the Salinas dataset. The CWG-SAGE method proposed by us has reached an astonishing 99.63%, 99.69%, and 99.59% on OA, AA, and Kappa, respectively.

3) Result Analysis for the WHU-Hi-HongHu Dataset: As shown in Table VI and Fig. 7, for the WHU-Hi-HongHu dataset, our proposed CWG-SAGE still achieves the best classification results, which is 1.20%, 2.31%, and 1.34% higher than the second best performing algorithm WFCG on OA, AA, and Kappa, respectively. In addition, CWG-SAGE achieved the best classification results in 13 out of 19 categories. Unlike the Indian Pines and Salinas datasets, in the WHU-Hi-HongHu dataset, the classification results of GNN-based methods are superior to all CNN-based methods. This is because the distribution structure of the ground cover of WHU Hi-HongHu is very large, and the test samples are very large, but the training samples are very few. This also shows the advantages of GNN-based methods in the case of insufficient training samples. When the proposed CWG-SAGE extracts pixel-level features through CNN, the CW-Conv and CAM modules pay more attention to the central features of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

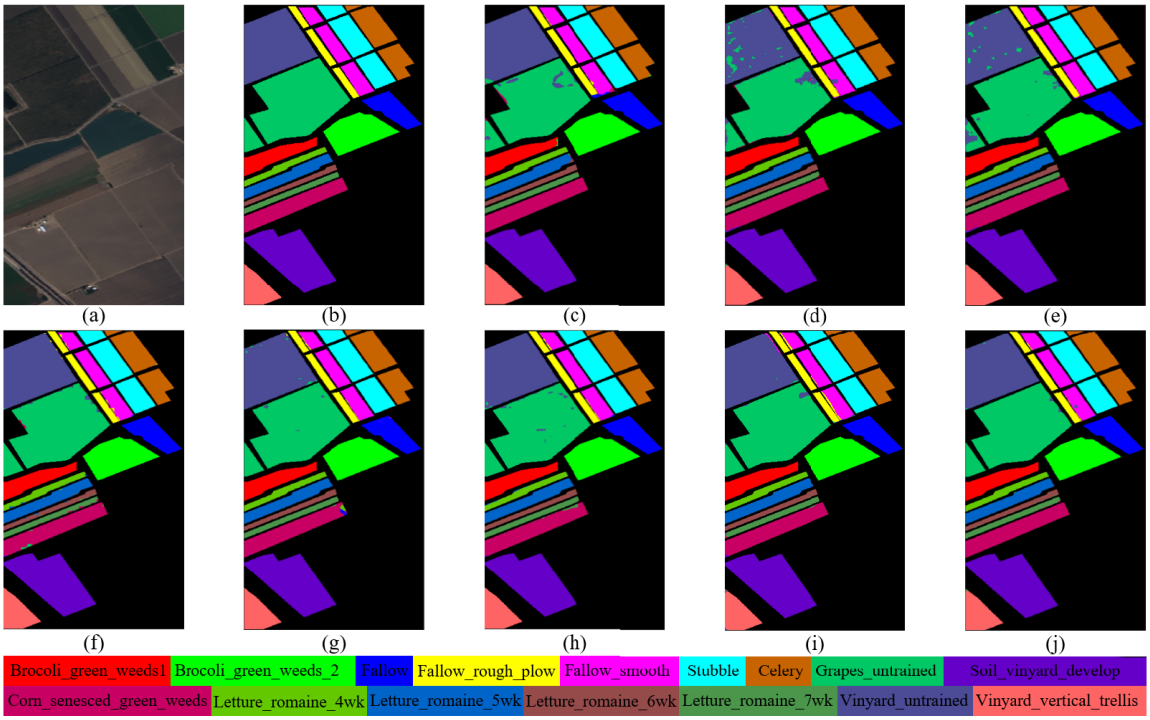


Fig. 6. Classification maps of different methods for the Salinas dataset. (a) False-color image. (b) Ground truth. (c) HybirdSN. (d) DBMA. (e) DBDA. (f) SAGE-A. (g) CEGCN. (h) WFCG. (i) FDGC. (j) CWG-SAGE.

TABLE VI

CLASSIFICATION RESULTS FOR THE WHU-HI-HIONGHU DATASET. BOLD NUMBERS INDICATE THE BEST PERFORMANCE IN THIS CATEGORY

Class	HybirdSN	DBMA	DBDA	SAGE-A	CEGCN	WFCG	FDGC	CWG-SAGE
1	89.68±2.657	93.84±1.681	94.30±1.629	95.68±3.086	96.65±2.251	97.03±2.245	**97.35±1.704**	95.57±2.486
2	38.46±12.123	81.85±6.277	85.65±4.528	70.29±4.830	84.98±4.835	84.25±7.171	73.55±4.626	**87.95±5.145**
3	95.01±1.361	96.61±1.506	96.25±1.044	98.14±0.502	98.83±0.549	97.88±0.723	96.547±0.351	**98.71±0.421**
4	99.60±0.265	99.70±0.127	99.60±0.176	99.46±0.346	99.78±0.083	99.62±0.071	99.78±0.085	**99.88±0.086**
5	99.66±0.327	99.53±0.054	**99.82±0.098**	98.88±0.347	99.60±0.072	99.60±0.142	99.36±0.496	99.72±0.243
6	89.43±5.635	90.82±1.703	93.55±2.435	96.06±0.499	95.20±1.610	95.27±2.058	**97.07±1.131**	96.91±1.397
7	35.68±16.943	58.65±3.103	64.34±5.886	96.45±1.957	82.99±6.459	92.25±3.829	87.55±7.772	**97.12±1.704**
8	99.62±0.203	99.55±0.082	99.53±0.028	98.34±0.480	99.35±0.297	99.63±0.197	99.72±0.237	**99.90±0.060**
9	88.08±7.888	94.64±0.913	94.45±3.385	94.27±0.767	96.45±0.967	96.87±1.279	95.79±1.645	**97.68±0.842**
10	**97.01±2.430**	95.63±0.437	94.41±3.189	94.09±1.952	95.01±1.235	94.72±1.703	95.56±2.771	96.98±0.968
11	77.86±1.378	76.11±2.006	80.50±5.482	92.64±2.593	92.45±1.260	92.43±1.792	92.81±3.238	**95.75±1.208**
12	91.55±3.099	91.87±1.499	95.34±1.232	94.55±1.576	96.03±1.921	94.86±0.552	96.34±1.435	**96.98±1.151**
13	81.79±10.415	88.75±0.972	93.48±1.559	94.53±0.911	93.84±1.045	95.09±1.948	95.35±1.127	**96.86±0.369**
14	37.02±25.134	82.36±6.082	91.68±4.124	95.84±1.247	86.93±3.687	84.77±1.874	93.65±5.371	**97.11±1.418**
15	94.39±1.938	95.11±1.362	97.01±0.984	95.73±1.407	97.22±0.768	**98.01±1.672**	97.45±1.861	97.19±1.653
16	83.60±3.918	91.53±1.731	89.34±3.211	90.98±1.778	96.13±2.118	94.31±4.910	**96.94±2.268**	95.34±2.919
17	83.95±5.940	90.23±2.123	92.12±4.592	91.18±3.508	93.20±3.977	93.67±0.991	92.54±2.859	**98.51±0.775**
18	85.23±11.395	91.50±0.935	91.57±1.742	90.44±1.614	91.92±2.002	91.78±1.328	93.61±1.719	**95.65±2.766**
19	76.68±7.335	89.39±4.428	92.42±1.037	95.61±3.650	96.09±2.510	95.94±1.309	90.83±5.083	**98.13±1.201**
OA(%)	91.23±1.459	93.86±0.346	95.10±0.605	96.31±0.115	96.84±0.315	96.88±0.352	96.81±0.401	**98.08±0.163**
AA(%)	81.22±3.487	89.88±0.568	91.86±0.887	93.85±0.256	94.35±0.518	94.63±0.447	94.31±1.125	**96.94±0.117**
Kappa	90.24±1.624	93.18±0.382	94.55±0.673	95.90±0.128	96.49±0.351	96.53±0.391	96.45±0.446	**97.87±0.181**

the samples, which can fully mine the sample information and make the network more stable, especially when dealing with small sample problems. Also, graphSAGE can reduce the spatial complexity of GCN under the condition of effectively aggregating node information

In general, the proposed CWG-SAGE can not only achieve smoother results in large feature structures but also retain fine information in small feature structures.

V. DISCUSSION

In this section, we design and conduct a series of experiments to analyze the effectiveness of CWG-SAGE from different perspectives and also verify the impact of different modules and hyperparameters on the results.

A. Influence of Label Ratio

We conduct experiments on these three datasets to investigate the classification accuracy performance of CWG-SAGE and other contrasting algorithms under a different number of labeled samples. The proportion of training samples for the Indian Pines dataset is set from 1% to 6% per class, the proportion of training samples for the Salinas dataset is set from 0.2% to 1.2% per class, and the proportion of training samples for the WHU-Hi-HongHu dataset is set from

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

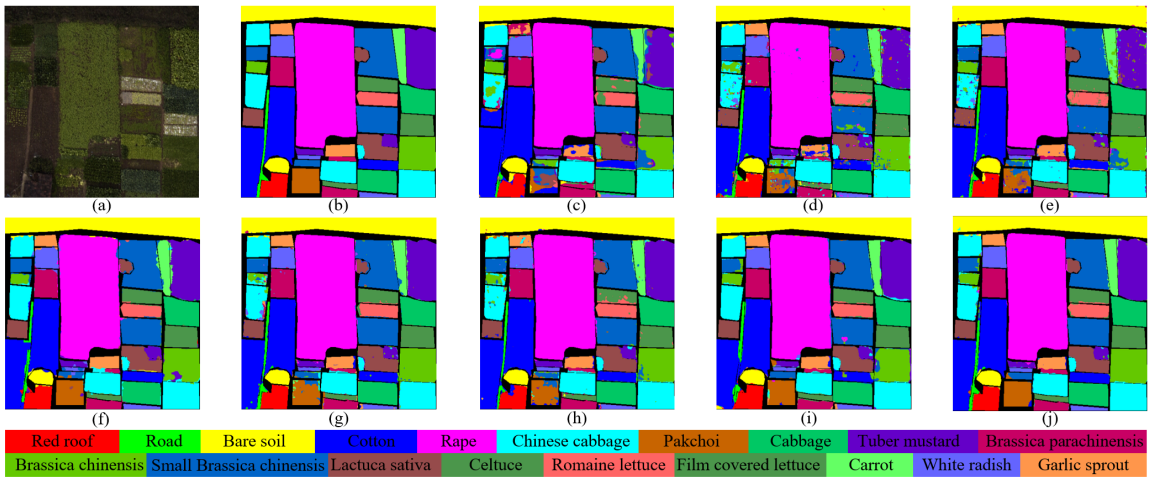


Fig. 7. Classification maps of different methods for the WHU-Hi-HongHu dataset. (a) False-color image. (b) Ground truth. (c) HybirdSN. (d) DBMA. (e) DBDA. (f) SAGE-A. (g) CEGCN. (h) WFCG. (i) FDGC. (j) CWG-SAGE.

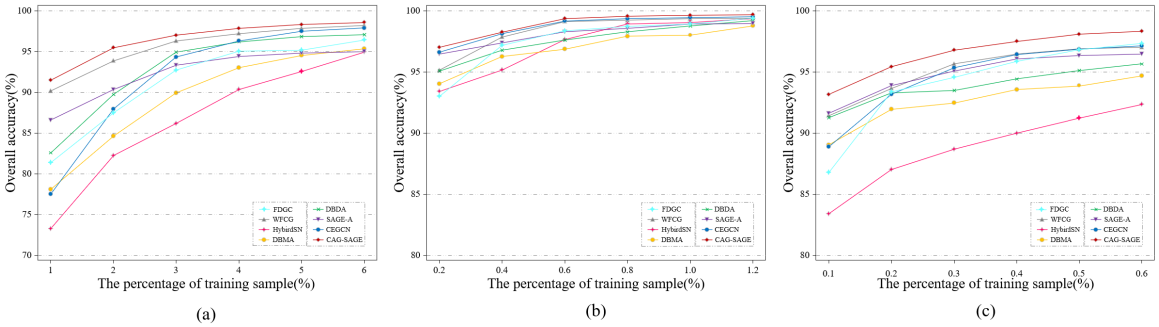


Fig. 8. Classification performance of each algorithm under different training ratios on three datasets. (a) Indian Pines. (b) Salinas. (c) WHU-Hi-HongHu.

0.1% to 0.6% per class; each experiment was repeated ten times and the results were averaged. The results are shown in Fig. 8; as expected, the proposed CWG-SAGE achieves the best classification accuracy on all three datasets. Since the labeling of HSI data samples consumes a lot of time and energy, the classification performance in the case of small samples can better test the quality of the algorithm. Even when there are few samples, our proposed CWG-SAGE can achieve very high accuracy, for example, in the Indian Pines dataset, under 1% training ratio, our CWG-SAGE's OA can reach 92%; and in the WHU-Hi-HongHu dataset, under 0.1% training ratio, our CWG-SAGE's OA can reach 93.5%, which is much higher than other algorithms. Overall, as the training ratio increases, the classification accuracy of each algorithm also increases. It should be noted that when the proportion of training samples is small, SAGE-A can achieve a high OA, but as the training proportion increases, it will gradually reach the upper limit and will be smaller than the OA obtained by the CNN-based method. This shows that the GNN-based method has a greater advantage in the case of small samples, and the CNN-based method has a greater advantage in the case of sufficient training samples.

B. Effectiveness of Different Modules in CWG-SAGE

In order to verify the effectiveness of each module in CWG-SAGE, we performed ablation experiments by replacing CW-Conv with ordinary convolution kernels, deleting CAM module, using only GraphSAGE branch, and using only CNN branch. Each experiment was repeated ten times, and the results were averaged, as shown in Fig. 9. We can clearly see that each module can improve the network performance to varying degrees. CW-Conv and CAM modules highlight the important position of the central pixel, improve the network feature extraction ability, make the network structure more stable, and improve the performance. The superpixel-based GraphSAGE can extract long-distance context information, which is complementary to the pixel-level features extracted by CNN. Through comparison, we can find

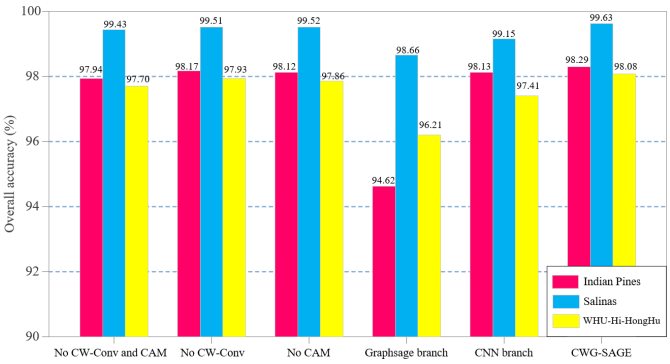
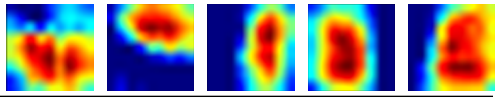
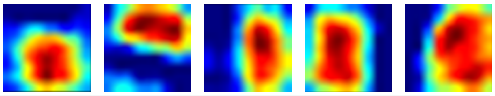


Fig. 9. Effectiveness of each module in CWG-SAGE.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
HEAT MAP VISUALIZATION OF LABEL MAP WITH AND WITHOUT CW-CONV AND CAM MODULES

Label map	
Without CW-Conv and CAM	
With CW-Conv and CAM	

that the performance improvement brought by the CW-Conv and CAM modules in the Salinas dataset is very small because this network framework has already achieved very good performance in this dataset. Not only that, by comparing without GraphSAGE and CWG-SAGE, we can also see that the performance improvement brought by the GraphSAGE module on the Indian Pines dataset is much smaller than that of the Salinas and WHU-Hi-HongHu datasets. The Salinas and WHU-Hi-HongHu datasets are much larger than Indian Pines, which shows that the operation of GraphSAGE using neighbor nodes to aggregate the features of the target node is more advantageous on large datasets.

In deep learning, CNN is always considered as a black box because it is difficult to describe its internal working principle in a reasonable way. Therefore, in recent years, in the field of computer vision, the interpretability of deep learning has gradually become a trend. Grad-CAM [47] is a powerful tool to explain the principle of deep learning. It uses the partial derivative as a weight factor to measure the contribution of each pixel to prediction and can capture the key areas of the image. In the previous paragraph, we quantitatively verified the performance improvement brought by CW-Conv and CAM. Here, we use Grad-CAM to visually observe the role of CW-Conv and CAM modules. The experimental results are shown in Table VII. In Table VII, the first row is the five label maps randomly selected by us, the second row is the heat maps without using the CW-Conv and CAM modules, and the third row is the heat maps after using the CW-Conv and CAM modules. The closer the color of the heap map is to the dark red, the greater the weight of this area to the pixel prediction. We can see that after using CW-Conv and CAM modules, the dark red area of the heap map is closer to the correct pixel in the center, especially the first, third, fourth, and fifth heap maps where dark red areas are significantly closer to the correct pixel in the center.

C. Influence of Segmentation Scale

The segmentation scale is the number of pixels in each superpixel, and larger segmentation scales result in larger superpixels that retain larger ground objects and fewer graph nodes. Conversely, smaller segmentation scales yield smaller superpixels that preserve smaller ground objects and more graph nodes.

The size of segmentation scales will affect the classification results of the CWG-SAGE, so we set different segmentation scales to test the classification accuracy. Considering the memory consumption of GPU, for the Indian Pines dataset, the segmentation scale is set to 100, 200, 300, 400, and 500; for the Salinas dataset, the segmentation scale is set to 100, 200, 300, 400, and 500; and for the WHU-Hi-HongHu dataset, the segmentation scale is set to 200, 300, 400, 500, and 600. Note that the segmentation scales of all three datasets in our other experiments are 100, 100, and 200. The classification results under different segmentation scales are shown in Fig. 10. For the Indian Pines dataset, the classification accuracy decreases with the increase of segmentation scale, which indicates that there are many small structures in this dataset. For the Salinas dataset, the classification accuracy increases with the increase of the segmentation scale, which indicates that there are many large structures in these two datasets. Due to the poor effect of superpixel segmentation, the classification accuracy of WHU-Hi-HongHu dataset under different segmentation scales does not show obvious regularity. The variance of OA does not change significantly with the change in segmentation scale, indicating that the network is stable.

D. Influence of Patch Size

In Section V-C, we analyzed the impact of segmentation scale on classification accuracy of our proposed CWG-SAGE. The impact of segmentation scale is mainly reflected in the region-level feature extraction branch. In this section, we analyze the influence of the input patch size in the pixel-level feature extraction branch on the classification results of CWG-SAGE. The patch size determines how many spatial domains are used to assist feature extraction. A larger patch size is beneficial to capture further contextual information, and conversely, a smaller patch size is beneficial to capture closer spatial contextual information. We validate the classification performance of CWG-SAGE with patch sizes ranging from 5 to 13, and the experimental results are shown in Table VIII. It can be seen from Table VIII that it is not that the larger the patch size, the higher the classification accuracy of CWG-SAGE. For the Indian Pines dataset, the classification accuracy is lowest when the patch size is 5, then becomes higher as the patch size increases and reaches a maximum at 9, and finally decreases as the patch size increases. The classification accuracy of Salinas dataset increases with the size of the patch and reaches 99.72% when the size is 13; this shows that when the ground structure of HSI is large, the large patch size has advantages. The WHU-Hi-HongHu dataset achieves the highest classification accuracy with a patch size of 11. Overall, when the patch size is set to 5, the classification accuracy performance on all three datasets is the worst because the patches are too small to extract enough features. When the patch size is set to 9, the sum of the classification accuracies of the three datasets performs the best. For the convenience of comparison, we use the patch size of 9 for the three datasets in all other experiments.

E. Analysis of Feature Fusion

Our proposed CWG-SAGE algorithm improves the classification performance by extracting features from two branches

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

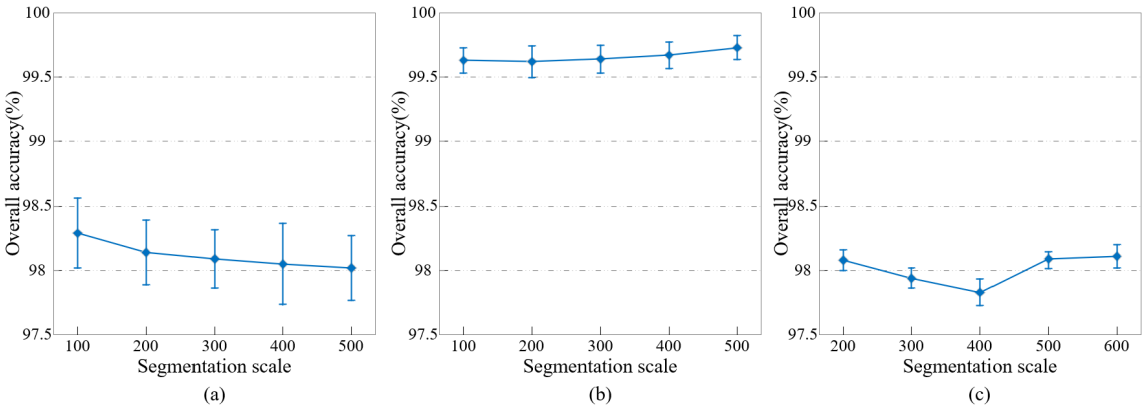


Fig. 10. Classification accuracy of different segmentation ratios on each dataset. (a) Indian Pines. (b) Salinas. (c) WHU-Hi-HongHu.

TABLE VIII

OAs OF CWG-SAGE NETWORK WITH DIFFERENT SIZES OF INPUT PATCHES. BOLD NUMBERS INDICATE THE BEST PERFORMANCE

Patch Size	5×5	7×7	9×9	11×11	13×13
Indian Pines	97.73±0.351	98.23±0.453	**98.29±0.542**	98.02±0.442	98.01±0.448
Salinas	99.39±0.252	99.52±0.183	99.63±0.154	99.67±0.107	**99.72±0.121**
WHU-Hi-HongHu	96.42±0.473	97.35±0.384	98.08±0.198	**98.24±0.198**	98.17±0.265

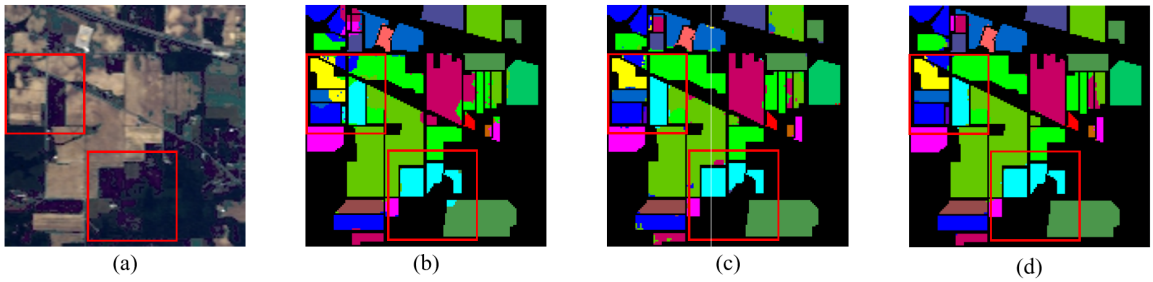


Fig. 11. Visualization of feature fusion on Indian Pines dataset. (a) False-color image. (b) Visualization of all areas by region-level feature extraction. (c) Visualization of all areas by pixel-level feature extraction. (d) Visualization of all areas by CWG-SAGE.

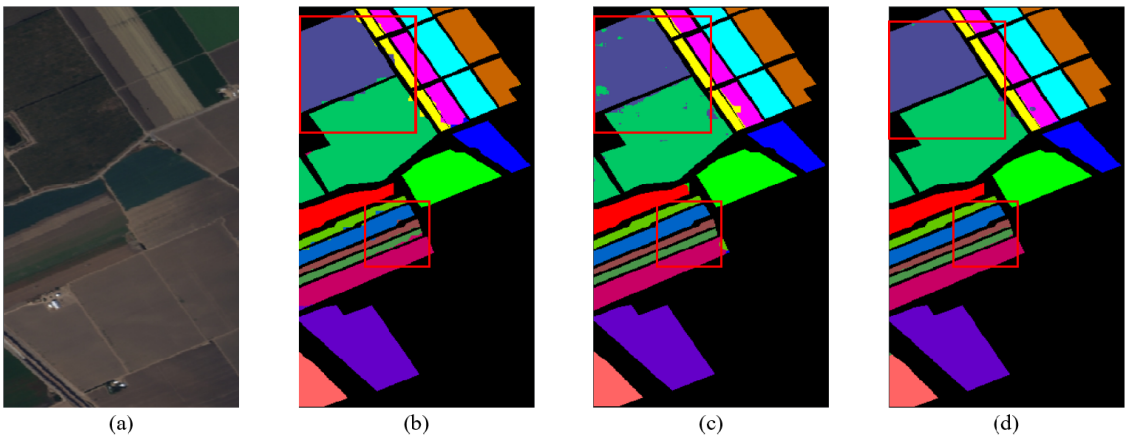
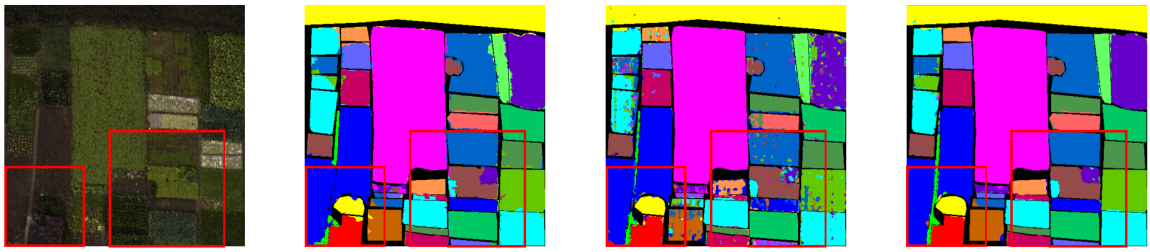


Fig. 12. Visualization of feature fusion on Salinas dataset. (a) False-color image. (b) Visualization of all areas by region-level feature extraction. (c) Visualization of all areas by pixel-level feature extraction. (d) Visualization of all areas by CWG-SAGE.

and then performing feature fusion. To verify the effectiveness of feature fusion, we visualize the results obtained by the region-pixel feature extraction branch, pixel-level feature extraction branch, and CWG-SAGE. Figs. 11–13 show the classification maps results of three datasets. Let us focus on the area in the red box, and we can see that the classification result of the superpixel-based region-level feature extraction is smoother, but the classification performance is not good on small areas. Pixel-level feature extraction can refine the structure of a small area and achieve good classification results on the structure of small areas, but it will produce salt and pepper noise in a relatively smooth large area. It can be seen

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d)

Fig. 13. Visualization of feature fusion on WHU-Hi-HongHu dataset. (a) False-color image. (b) Visualization of all areas by region-level feature extraction. (c) Visualization of all areas by pixel-level feature extraction. (d) Visualization of all areas by CWG-SAGE.

TABLE IX

OAs OF CWG-SAGE NETWORK WITH DIFFERENT FEATURE FUSION STRATEGIES. BOLD NUMBERS INDICATE THE BEST PERFORMANCE

Fusion Strategy	Indian Pines	Salinas	WHU-Hi-HongHu
Addition (A)	97.52±0.365	99.53±0.213	97.87±0.217
Multiplication (M)	96.98±0.459	99.39±0.326	95.68±0.401
Concatenation (C)	**98.29±0.542**	**99.63±0.154**	**98.08±0.163**

from the results that feature fusion can combine the advantages of the two branches to improve the classification performance of CWG-SAGE.

Three fusion strategies are considered in this article: addition (A), elementwise multiplication (M), and concatenation (C). We sequentially use these three fusion strategies to conduct experiments on three datasets and use OA to measure the performance of different fusion strategies. The experimental results are shown in Table IX. All three datasets achieve the highest classification accuracy under fusion strategy C and all achieve the worst classification accuracy under fusion strategy M. It should be noted that the Salinas dataset can achieve high classification accuracy under all three fusion strategies, and the classification results of the Indian Pines and WHU-Hi-HongHu datasets under the fusion strategy M are extremely poor.

VI. CONCLUSION

In this article, a novel network CWG-SAGE for HSI classification is proposed. CWG-SAGE extracts long-range contextual information through a graph neural network GraphSAGE in a region-level feature extractor and pixel-level features through a CNN in a pixel-level feature extractor. Specifically, pixel-level features are mainly extracted through the spatial–spectral feature block. When extracting pixel-level features, we suggest using the proposed CAM module and using CW-Conv instead of ordinary convolution kernel to enhance the position of data center pixels so that the network can independently learn the primary and secondary relationship between central features and edge features, thus improving the feature extraction ability and network stability; in the region-level feature extractor, we use a graph neural network to extract the long-range contextual information of the sample. Obtaining superpixels through SLIC image segmentation and then building a graph neural network GraphSAGE through a graph encoder and a graph decoder, we can overcome the disadvantage of high computational cost of the graph CNN. Through a series of experiments, it also proved that our proposed CWG-SAGE has higher robustness and stability and can achieve very high classification accuracy in the case of insufficient training samples; in addition, it can also achieve very high accuracy in the case of categories with fewer samples. Next, we will also explore more advanced network structures such as a hybrid architecture that combines GNN and CNN in one channel to take full advantage of the rich spectral information contained in HSI.

REFERENCES

[1] P. Ghamisi, J. Plaza, Y. Chen, J. Li, and A. J. Plaza, "Advanced spectral classifiers for hyperspectral images: A review," *IEEE Geosci. Remote Sens. Mag.*, vol. 5, no. 1, pp. 8–32, Mar. 2017.

[2] M. Govender, K. Chetty, and H. Bulcock, "A review of hyperspectral remote sensing and its application in vegetation and water resource studies," *Water SA*, vol. 33, no. 2, pp. 145–151, Dec. 2009.

[3] A. Ghiyamat and H. Z. Shafri, "A review on hyperspectral remote sensing for homogeneous and heterogeneous forest biodiversity assessment," *Int. J. Remote Sens.*, vol. 31, no. 7, pp. 1837–1856, 2010.

[4] G. Lu and B. Fei, "Medical hyperspectral imaging: A review," *J. Biomed. Opt.*, vol. 19, no. 1, Jan. 2014, Art. no. 010901.

[5] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[6] H. Zhang, J. Li, Y. Huang, and L. Zhang, "A nonlocal weighted joint sparse representation classification method for hyperspectral imagery," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2056–2065, Jun. 2014.

[7] H. Yuan, Y. Y. Tang, Y. Lu, L. Yang, and H. Luo, "Hyperspectral image classification based on regularized sparse representation," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2174–2182, Jun. 2014.

[8] D. Landgrebe, "Hyperspectral image data analysis," *IEEE Signal Process. Mag.*, vol. 19, no. 1, pp. 17–28, Jan. 2002.

[9] K. Makantasis, K. Karantzalos, A. Doulamis, and N. Doulamis, "Deep supervised learning for hyperspectral data classification through convolutional neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2015, pp. 4959–4962.

[10] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997.

[11] S. K. Roy, J. M. Haut, M. E. Paoletti, S. R. Dubey, and A. Plaza, "Generative adversarial minority oversampling for spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2021, Art. no. 5500615.

[12] R. Hang, F. Zhou, Q. Liu, and P. Ghamisi, "Classification of hyperspectral images via multitask generative adversarial networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 2, pp. 1424–1436, Feb. 2021.

[13] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

[14] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[15] S. K. Roy, S. Manna, T. Song, and L. Bruzzone, "Attention-based adaptive spectral–spatial kernel Resnet for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 9, pp. 7831–7843, Sep. 2021.

[16] J. Yang, Y.-Q. Zhao, and J. C.-W. Chan, "Learning and transferring deep joint spectral–spatial features for hyperspectral classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 8, pp. 4729–4742, Aug. 2017.

[17] Y. Cui, W. Li, L. Chen, S. Gao, and L. Wang, "Double-branch local context feature extraction network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[18] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[19] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, Jun. 2018, pp. 7132–7141.

[20] J. Fu et al., "Dual attention network for scene segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 3141–3149.

[21] W. Ma, Q. Yang, Y. Wu, W. Zhao, and X. Zhang, "Double-branch multi-attention mechanism network for hyperspectral image classification," *Remote Sens.*, vol. 11, no. 11, p. 1307, 2019.

[22] S. Woo, J. Park, J. Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Sep. 2018, pp. 3–19.

[23] R. Li, S. Zheng, C. Duan, Y. Yang, and X. Wang, "Classification of hyperspectral image based on double-branch dual-attention mechanism network," *Remote Sens.*, vol. 12, no. 3, p. 582, Feb. 2020.

[24] J. He, L. Zhao, H. Yang, M. Zhang, and W. Li, "HSI-BERT: Hyperspectral image classification using the bidirectional encoder representation from transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 1, pp. 165–178, Sep. 2020.

[25] Y. Cui, J. Xia, Z. Wang, S. Gao, and L. Wang, "Lightweight spectral–spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–14, 2021.

[26] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1314–1324.

[27] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Represent.*, 2017, pp. 1–14.

[28] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[29] A. Qin, Z. Shang, J. Tian, Y. Wang, T. Zhang, and Y. Y. Tang, "Spectral–spatial graph convolutional networks for semisupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 2, pp. 241–245, Feb. 2019.

[30] S. Wan, C. Gong, P. Zhong, B. Du, L. Zhang, and J. Yang, "Multiscale dynamic graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3162–3177, May 2020.

[31] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "CNN-enhanced graph convolutional network with pixel- and superpixel-level feature fusion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8657–8671, Oct. 2021.

[32] Y. Dong, Q. Liu, B. Du, and L. Zhang, "Weighted feature fusion of convolutional neural network and graph attention network for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 31, pp. 1559–1572, 2022.

[33] Q. Liu, Y. Dong, Y. Zhang, and H. Luo, "A fast dynamic graph convolutional network and CNN parallel network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5530215.

[34] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, Long Beach, CA, USA, 2017, pp. 1025–1035.

[35] P. Yang, L. Tong, B. Qian, Z. Gao, J. Yu, and C. Xiao, "Hyperspectral image classification with spectral and spatial graph using inductive representation learning network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 791–800, Dec. 2021.

[36] Y. Ding, X. Zhao, Z. Zhang, W. Cai, and N. Yang, "Multiscale graph sample and aggregate network with context-aware learning for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 4561–4572, 2021.

[37] Y. Ding, X. Zhao, Z. Zhang, W. Cai, and N. Yang, "Graph sample and aggregate-attention network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[38] D. Hong, N. Yokoya, J. Chanussot, and X. X. Zhu, "CoSpace: Common subspace learning from hyperspectral-multispectral correspondences," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 4349–4359, Jul. 2019.

[39] D. Hong, N. Yokoya, N. Ge, J. Chanussot, and X. X. Zhu, "Learnable manifold alignment (LeMA): A semi-supervised cross-modality learning framework for land cover and land use classification," *ISPRS J. Photogramm. Remote Sens.*, vol. 147, pp. 193–205, Jan. 2019.

[40] D. K. Hammond, P. Vandergheynst, and R. Gribonval, "Wavelets on graphs via spectral graph theory," *Appl. Comput. Harmon. Anal.*, vol. 30, no. 2, pp. 129–150, Mar. 2011.

[41] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.

[42] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Mach. Learn.*, vol. 37. Lille, France, 2015, pp. 448–456.

[43] X. Ding, Y. Guo, G. Ding, and J. Han, "ACNet: Strengthening the kernel skeletons for powerful CNN via asymmetric convolution blocks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1911–1920.

[44] Y. Zhong, X. Hu, C. Luo, X. Wang, J. Zhao, and L. Zhang, "WHU-Hi: UAV-borne hyperspectral with high spatial resolution (H^2) benchmark datasets and classifier for precise crop identification based on deep convolutional neural network with CRF," *Remote Sens. Environ.*, vol. 250, Dec. 2020, Art. no. 112012.

[45] Y. Zhong et al., "Mini-UAV-borne hyperspectral remote sensing: From observation and processing to applications," *IEEE Geosci. Remote Sens. Mag.*, vol. 6, no. 4, pp. 46–62, Dec. 2018.

[46] R. O. Green et al., "Imaging spectroscopy and the airborne visible/infrared imaging spectrometer (AVIRIS)," *Remote Sens. Environ.*, vol. 65, no. 3, pp. 227–248, Sep. 1998.

[47] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," *Int. J. Comput. Vis.*, vol. 2017, pp. 618–626, Dec. 2017.

Ying Cui received the Ph.D. degree in signal and information processing and the Ph.D. degree in computer science and technology from Harbin Engineering University (HEU), Harbin, China, in 2006 and 2009, respectively.

She is currently an Associate Professor at the School of Information and Communication Engineering, HEU. Her research interests include intelligent signal processing, image processing, and wireless sensor network optimization.

Chao Shao received the B.E. degree in communication engineering from Xidian University (XDU), Xi'an, China, in 2020. He is currently pursuing the M.S. degree in information and communication engineering with Harbin Engineering University (HEU), Harbin, China.

His research interests include deep learning, computer vision, and remote sensing image processing.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.

Li Luo received the B.E. degree in communication engineering from Harbin Engineering University (HEU), Harbin, China, in 2022, where she is currently pursuing the M.S. degree in information and communication engineering.

Her research interests include deep learning, computer vision, and remote sensing image processing.

Liguo Wang (Member, IEEE) received the M.S. degree in natural science and the Ph.D. degree in engineering from the Harbin Institute of Technology (HIT), Harbin, China, in 2002 and 2006, respectively.

He is currently a Professor at the School of Information and Communication Engineering, Dalian Minzu University, Dalian, China. He has published four books, 35 patents, and more than 230 papers in journals and conference proceedings. His research interests include remote sensing image processing and machine learning.

Shan Gao received the B.S. degree in electronic information engineering and the M.S. degree in communication and information systems from Harbin Engineering University (HEU), Harbin, China, in 2009 and 2012, respectively, and the Ph.D. degree from the School of Electrical Engineering and Automation, Harbin Institute of Technology, Harbin, in 2017.

He is currently an Assistant Professor with the College of Information and Communication Engineering, HEU. His research interests include optical detection technology, artificial intelligence, and engine measurement technology.

Liwei Chen received the Ph.D. degree in electronic information engineering and the Ph.D. degree in signal and information processing from Harbin Engineering University, Harbin, China, in 1998 and 2005, respectively.

She is currently an Associate Professor at the School of Information and Communication Engineering, Harbin Engineering University. She has authorized five invention patents and published over 40 articles. Her research interests include artificial intelligence, deep learning, image segmentation, feature extraction, classification recognition, and understanding.

Dr. Chen is a member of the China Society of Image and Graphics.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 15:46:47 UTC from IEEE Xplore. Restrictions apply.